PROJECT PROPOSAL

Silent Disco

A browser-based system for synchronized multi-device audio playback using Express, WebSockets, clock synchronization and the Web Audio API

Student name	[Your full name]
University seat number (USN)	[Your USN]
Programme / Semester	B.E. Final year (4th year) / [Semester]
Department	[Department name]
Institution	[College name]
Project guide	[Teacher / guide name]
Project type	Individual full-stack web application
Date of submission	09 October 2026
Implementation status	Backend and frontend written; integration testing and accuracy measurements pending (see Section 14)

Table of Contents

1. Abstract 4

2. Problem Statement and Motivation 4

2.1 Why it is a good final-year project . . . 4

3. Objectives 4

4. Scope 5

4.1 In scope . . . 5

4.2 Out of scope (deliberately) . . . 5

5. Background and Related Work 5

5.1 Clock synchronization . . . 5

5.2 Browser audio timing . . . 5

5.3 Real-time transport . . . 5

5.4 Existing approaches and why they were not chosen . . . 5

6. Requirements 6

6.1 Functional requirements . . . 6

6.2 Non-functional requirements . . . 6

7. System Architecture 6

7.1 Technology stack . . . 7

7.2 Repository layout . . . 7

8. Data Model 7

8.1 Playback state semantics . . . 8

9. Interface Specification 8

9.1 REST API . . . 8

9.2 WebSocket protocol . . . 8

10. Clock Synchronization and Playback Algorithm 9

10.1 Estimating the server clock offset . . . 9

10.2 Scheduling playback . . . 9

10.3 Worked example . . . 10

10.4 Drift and re-synchronization . . . 10

10.5 Manual latency compensation . . . 10

10.6 Expected accuracy and its limits . . . 10

11. Frontend Design 10

11.1 Client state and flow . . . 10

12. Security, Privacy and Abuse Prevention 11

13. Deployment Plan 11

14. Testing and Evaluation Plan 11

14.1 Functional test cases . . . 11
14.2 Synchronization accuracy experiment . . . 12
14.3 Other checks . . . 12
15. Risks, Limitations and Mitigations **13**
16. Work Plan **13**
16.1 Initial build (one intensive session) . . . 13
16.2 Evaluation and report phase . . . 13
17. Expected Deliverables and Learning Outcomes **13**
Deliverables . . . 13
Learning outcomes . . . 14
18. Future Enhancements **14**
19. References **14**
Appendix A. Running the Project Locally **14**
Appendix B. Demo Script (about 3 minutes) **14**

1. Abstract

Silent Disco is a full-stack web application that lets a group of people listen to the same song, at the same moment, on their own phones, with no speaker and no app installation. One person creates a party room, shares a link, and uploads an audio file. Every device that joins downloads the file, estimates the offset between its own clock and the server clock, and then schedules playback with sample-accurate timing using the Web Audio API. The host controls play, pause and seek; all listeners follow.

The project combines three areas of computer science that students usually study in isolation: networking (HTTP, WebSockets, round-trip time), distributed systems (clock synchronization, shared state, ordering) and web engineering (REST APIs, browser audio pipelines, responsive UI). The entire system is a single Node.js project in which one Express server provides the REST API, serves the frontend, and hosts the WebSocket endpoint, so it can be built, run and deployed as one unit.

Keywords: clock synchronization, Cristian's algorithm, WebSocket, Web Audio API, Express.js, real-time systems, distributed playback.

2. Problem Statement and Motivation

Shared listening today relies on speakers (which disturb others in libraries, hostels, trains and shared rooms) or on streaming-service group sessions (which need accounts, paid subscriptions and the same app on every device). A simple, free, no-install alternative does not exist for a casual group such as a hostel floor, a study group or friends on a trip.

The technical difficulty is that independent devices have independent clocks. If each phone simply starts playing when a message arrives, network jitter of even 100 to 200 ms makes the audio sound like an echo. The problem therefore reduces to: *how can several browsers, each with an unknown clock offset and unknown network delay, start and keep playing one audio track close enough in time that a listener perceives them as simultaneous?*

This project answers that question with a practical, measurable and well-documented implementation.

2.1 Why it is a good final-year project

- Non-trivial engineering core (clock sync) rather than a CRUD application.
- Visible, demo-friendly result: the teacher can hear it working on two phones in the room.
- Measurable outcome: synchronization error can be experimentally quantified and reported.
- Covers a full stack: REST API, WebSockets, static hosting, browser audio, deployment.
- Deployable on free infrastructure and usable by real people immediately.

3. Objectives

- Design and implement a room-based system where one host controls playback for many listeners.
- Implement application-level clock synchronization over WebSockets using round-trip measurements and lowest-RTT sample selection.
- Schedule audio start times with the Web Audio API clock rather than relying on HTMLAudioElement.play().
- Keep devices in sync during long playback using periodic re-synchronization and drift correction.
- Support late joiners (join mid-song at the correct position) and automatic reconnection.
- Provide a manual latency compensation control for Bluetooth headphones.

- Build everything inside one Express-based project (API, static frontend, WebSocket) with no external database for the base version.
- Evaluate synchronization accuracy experimentally and document the method and results.
- Deploy the application on a public HTTPS URL and document setup, limitations and future work.

4. Scope

4.1 In scope

- Create room, join by code or link, live listener and readiness counts.
- Host-only audio upload (audio files up to 25 MB) and host-only play, pause and seek.
- Distribution of the uploaded file to all listeners over HTTP.
- Clock synchronization, scheduled playback, drift correction, per-device delay slider.
- Rate limiting, input validation and host authorization through a secret token.
- Responsive single-page UI that works on phone browsers.

4.2 Out of scope (deliberately)

- Streaming-service integration (Spotify, YouTube). Their SDKs and embeds cannot be scheduled to millisecond accuracy and have licensing and account restrictions.
- User accounts, persistent playlists, room history and analytics (no database in version 1).
- Guaranteed sub-millisecond synchronization. Browsers and networks do not allow it.
- Automatic detection of Bluetooth latency (not exposed to web pages); handled manually.
- Native mobile applications.

5. Background and Related Work

5.1 Clock synchronization

Network Time Protocol (NTP, RFC 5905) and Cristian's algorithm (Cristian, 1989) estimate a remote clock by measuring a request and its reply. If the client records the send time t0 and receive time t1, and the server reports its time ts in between, the client assumes the reply was stamped halfway through the round trip. The estimate's maximum error is half the round-trip time (RTT), which is why selecting the sample with the smallest RTT gives the best estimate. Silent Disco applies this idea at application level over a WebSocket.

5.2 Browser audio timing

The Web Audio API exposes AudioContext.currentTime, a high-precision clock driven by the audio hardware, and AudioBufferSourceNode.start(when, offset), which starts a decoded buffer at a precise future time and from a chosen position in the buffer. Compared with HTMLAudioElement.play(), whose start time depends on decoding, buffering and scheduling delays, this gives far tighter control.

5.3 Real-time transport

WebSocket (RFC 6455) provides a persistent, low-overhead, bidirectional channel over a single TCP connection, which suits small frequent control messages such as ping/pong and playback state updates. Large payloads (the audio file) are sent over normal HTTP, which supports standard caching, size limits and error handling.

5.4 Existing approaches and why they were not chosen

Approach	Limitation for this use case
Streaming-app group sessions	Need accounts, often paid, same app on every device; not an engineering project.
YouTube / Spotify embeds synced by messages	Start time cannot be scheduled precisely; ads, buffering and DRM vary per device.
Dedicated multi-room audio software	Needs installation and a local network; not browser based.
Plain HTML audio with a broadcast play message	Each device starts at a different moment due to network jitter; audible echo.

6. Requirements

6.1 Functional requirements

ID	Requirement
FR1	A user can create a room and receives a 6-character room code and a secret host token.
FR2	Any user can join an existing room using the code or a shared link.
FR3	Only the host can upload audio and issue play, pause and seek commands.
FR4	On upload, every connected client downloads and decodes the track and reports ready.
FR5	The host's Play button is enabled only when every connected client is ready.
FR6	All clients start playback at the same shared server-time instant.
FR7	Clients re-synchronize their clock every 10 seconds and correct drift above 60 ms.
FR8	A client that joins while a song is playing joins at the correct position.
FR9	A client that loses its connection reconnects automatically and re-synchronizes.
FR10	Each user can apply a manual delay compensation between -300 and +300 ms.
FR11	Rooms and uploaded audio are deleted after the room has been empty for 60 seconds.

6.2 Non-functional requirements

Category	Requirement
Performance	Perceived synchronization within tens of milliseconds on typical Wi-Fi/4G (to be verified experimentally; target to be reported in Section 14).
Usability	Works in current mobile Chrome and Safari with a single tap to join; no install.
Security	Host actions authorized by an unguessable token; uploads validated by type and size; creation rate limited.
Privacy	No accounts, no personal data stored; audio kept only in memory.
Reliability	Graceful handling of disconnects, missing rooms, failed downloads, bad files.
Maintainability	Single repository, no build step, small readable modules, documented.
Portability	Runs on Node.js 18 or newer on any host that supports long-lived connections.

7. System Architecture

The system is one Node.js process. Express serves three roles: the REST API under /api, the static frontend from the public folder, and (through the ws library attached to the same HTTP server) the WebSocket endpoint at /ws. Room state is held in a JavaScript Map in memory.

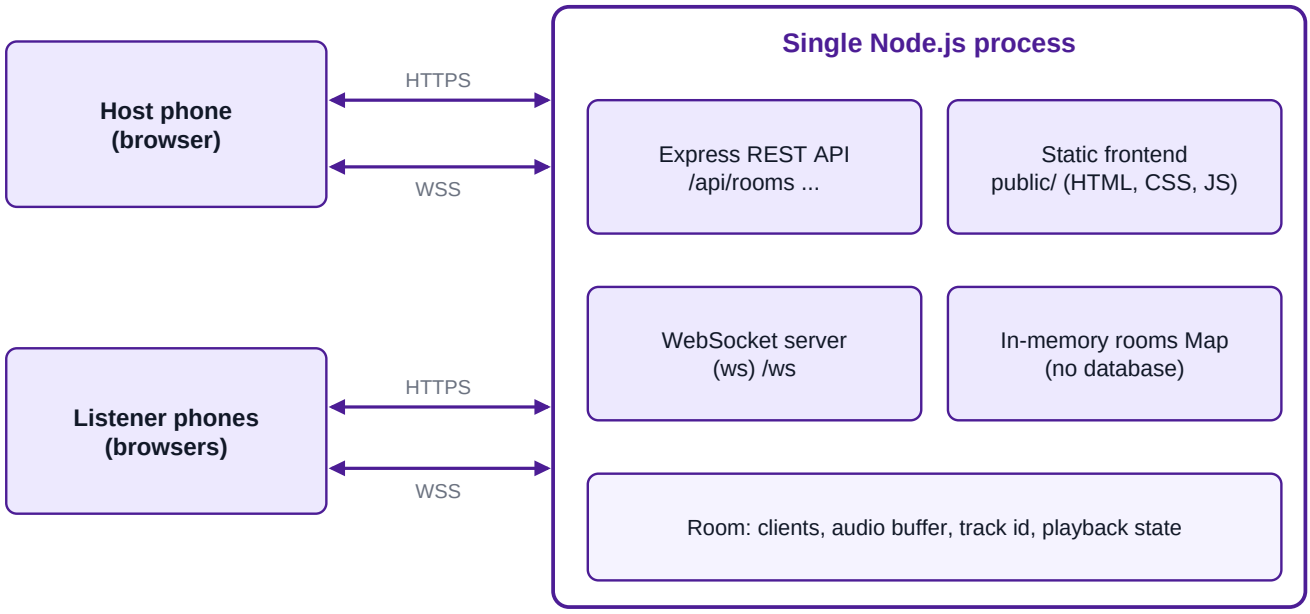

Figure 1. High-level architecture.

7.1 Technology stack

Layer	Technology	Reason
Runtime	Node.js 18+	Single language across the stack; native async I/O suits many idle connections.
HTTP server / API	Express 4	Minimal, well known; serves API and static files from one app.
Real-time	ws 8 (WebSocket)	Lightweight, standards-compliant, shares the HTTP server.
Frontend	HTML, CSS, vanilla JavaScript	No build step; keeps focus on the sync logic; small payload.
Audio	Web Audio API	Hardware-driven clock and scheduled start of decoded buffers.
Storage	In-memory Map (no database)	Rooms are temporary; simplest correct design.
Hosting	Render / Railway / Fly.io (one web service)	Supports persistent WebSocket connections and HTTPS.

7.2 Repository layout

```
silent-disco/
  package.json        dependencies (express, ws) and scripts
  server/
    index.js          Express API + static hosting + WebSocket server
  public/
    index.html        landing, join gate and room screens
    style.css         responsive dark UI
    app.js            clock sync, Web Audio scheduling, UI logic
  docs/
    Silent-Disco-Project-Proposal.pdf
```

8. Data Model

There is no database. The server keeps one object per active room in a Map keyed by the room code.

Field	Type	Meaning
code	string (6 hex chars)	Public room identifier, from 3 random bytes (about 16.7 million combinations), re-drawn on collision.

Field	Type	Meaning
hostToken	string (32 hex chars)	Secret from 16 random bytes; whoever presents it on the WebSocket is the host.
clients	Set of WebSocket	Connected sockets; each socket has isHost and ready flags.
audio	Buffer or null	Raw bytes of the uploaded track, kept in memory only.
mime	string	Content type of the audio, returned on download.
name	string (max 120 chars)	Original file name shown to listeners.
trackId	integer	Incremented per upload so clients know when to reload.
state	object	Shared playback state: playing (bool), position (seconds), serverTime (ms, server clock).
timer	timeout handle	Deletes the room 60 s after the last client leaves.

8.1 Playback state semantics

The state object means: *at server time serverTime, the track is (or was) at position seconds, and it is playing if playing is true.* When the host presses Play, the server sets serverTime to now plus 1000 ms. This lead time gives every client time to receive the message and schedule the start in the future. Pause records the exact position and the current time. Because the state is a statement about the server clock, any client can compute the correct position at any later moment, which is what makes late joining and drift correction possible.

9. Interface Specification

9.1 REST API

Method and path	Purpose	Request	Success	Errors
POST /api/rooms	Create a room	none	201 {code, hostToken}	429 if more than 10 rooms per IP per hour
GET /api/rooms/:code	Check a room exists	none	200 {code, listeners, hasTrack}	404 room not found
POST /api/rooms/:code/audio	Host uploads track	Raw audio body; headers Content-Type: audio/*, X-Host-Token, X-File-Name	200 {ok, trackId}; broadcasts track to the room	403 bad token; 404; 415 not audio; 400 empty; 413 over 25 MB
GET /api/rooms/:code/audio	Listeners download track	none	200 audio bytes, no-store	404 no audio
GET /api/health	Liveness check	none	200 {ok, rooms}	none

9.2 WebSocket protocol

Connection URL: /ws?room=CODE&token=HOSTTOKEN. The token is optional; if it matches the room's host token the socket is the host. An unknown room closes the socket with code 4004. All messages are JSON, limited to 4096 bytes.

Message	Direction	Fields	Effect
hello	server to client	isHost, track, state	Sent once on connect; client loads the track if present.
ping	client to server	t0	Starts one clock-sync sample.

Message	Direction	Fields	Effect
pong	server to client	t0, ts	Reply carrying the server clock.
ready	client to server	ready (bool)	Reports download and decode finished.
presence	server to all	listeners, ready	Updates counts; enables Play when equal.
track	server to all	track {id, name}, state	A new file was uploaded; reload it.
play	host to server	position	Server sets state.serverTime to now + 1000 ms and broadcasts state.
pause	host to server	position	Server stores position and broadcasts state.
seek	host to server	position	Server updates position and broadcasts state.
state	server to all	state {playing, position, serverTime}	Clients schedule or stop playback.

Example messages:

```
client -> server   {"type":"ping","t0":1760031000123}
server -> client   {"type":"pong","t0":1760031000123,"ts":1760031000187}
host   -> server   {"type":"play","position":12.4}
server -> all      {"type":"state","state":{"playing":true,"position":12.4,
                    "serverTime":1760031042500}}
```

10. Clock Synchronization and Playback Algorithm

This section is the technical core of the project.

10.1 Estimating the server clock offset

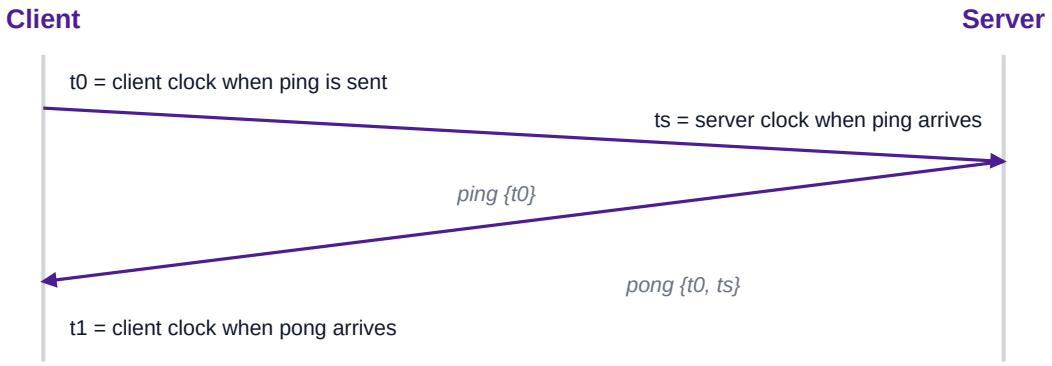

Figure 2. One ping/pong sample.

For every sample the client computes:

```
RTT     = t1 - t0
offset  = ts + RTT/2 - t1          (server clock minus client clock)
serverNow() = Date.now() + offset
```

Eight samples are sent 100 ms apart. The sample with the smallest RTT is used, because the error of the estimate is bounded by RTT/2 and a short round trip is the least likely to have been delayed asymmetrically in one direction. The Sync pill in the UI shows this bound (RTT/2) in milliseconds.

10.2 Scheduling playback

When a state message arrives, the client converts the shared state into a local action:

```
elapsed = (serverNow() - state.serverTime)/1000 + manualDelay/1000
if not state.playing:  stop, remember position
else if elapsed >= 0:  start NOW at  position + elapsed   (already started)
else:                  start in (-elapsed) seconds at position (future start)
```

```
source.start(audioCtx.currentTime + delay, startOffset)
```

The same rule covers every case: a normal Play (start is about one second in the future), a late joiner (start already in the past, so jump into the song), a seek, and a reconnect.

10.3 Worked example

Suppose a phone measures RTT = 40 ms and t1 = 5000 on its own clock, and the pong carried ts = 98020. Then offset = 98020 + 20 - 5000 = 93040. The host presses Play at position 0 and the server sets serverTime = 100000. When the state arrives at local time 5300, serverNow() = 98340, so elapsed = (98340 - 100000)/1000 = -1.66 s. The phone therefore schedules the start 1.66 seconds in the future on the audio clock; another phone with a different offset computes its own delay but starts at the same real-world instant.

10.4 Drift and re-synchronization

Every 10 seconds each client runs another sample burst. About 1.2 seconds later it compares its actual playback position (from the audio clock) with the position the shared state predicts. If the difference exceeds 60 ms, it re-applies the state, which restarts the source at the correct position. This bounds long-term drift caused by clock-rate differences between devices.

10.5 Manual latency compensation

Bluetooth headphones add roughly 100 to 300 ms of playback delay that a web page cannot measure. The delay slider (-300 to +300 ms) adds a user-chosen amount to elapsed so that the device plays slightly earlier or later; changing it re-applies the current state.

10.6 Expected accuracy and its limits

- Clock estimate error is at most half the smallest measured RTT (for example 20 ms for a 40 ms RTT, usually much lower in practice).
- Audio output latency differs per device and per output route (speaker, wired, Bluetooth). It is not part of the network estimate and is the reason for the manual slider.
- Restarting a source to correct drift may cause a brief audible glitch; the 60 ms threshold avoids needless corrections.
- These figures are design expectations. The actual measured values will be reported in Section 14.

11. Frontend Design

The frontend is a single page in plain HTML, CSS and JavaScript, served by Express. There are three screens controlled by showing or hiding sections.

Screen	Elements	Behaviour
Landing	Start a party button; room code field and Join button	Create calls POST /api/rooms and enters the room as host. Join checks the code with GET /api/rooms/:code first.
Join gate	Message with room code and Join the party button	Shown when the page is opened through a shared link. The tap creates and resumes the AudioContext, which mobile browsers require before audio can play.
Room	Room code, Copy link, status pills (connection, listeners, ready, sync accuracy), track name, host controls, delay slider, error area	Host sees file chooser, Play, Pause and seek bar. Guests see a note. Play is disabled until everyone is ready.

11.1 Client state and flow

- **Join:** tap creates the AudioContext, opens the WebSocket and starts a sync burst.
- **Hello:** server tells the client whether it is host and which track (if any) is loaded.
- **Load track:** fetch the file, decodeAudioData, send ready, then apply the current state.
- **Presence:** counts update; the host's Play button enables when ready equals listeners.
- **State:** every play, pause or seek is converted to scheduled local playback (Section 10.2).
- **Reconnect:** on socket close the client retries every 2 seconds, then re-syncs and reloads state.
- **Host recovery:** the host token is kept in localStorage per room so a page reload keeps host rights.

12. Security, Privacy and Abuse Prevention

Risk	Control in this design
Unauthorized control of playback	Host token (128 bits of randomness) required on the WebSocket and on upload; play, pause and seek are ignored from non-host sockets.
Room guessing	6-hex-character codes; joining only exposes listening, not control. Codes live only while a room exists.
Oversized or wrong uploads	25 MB limit; content type must start with audio/; empty bodies rejected.
Room creation spam	Maximum 10 rooms per IP per hour (HTTP 429). On a platform behind a reverse proxy, Express 'trust proxy' must be enabled so the real client IP is used.
WebSocket flooding	maxPayload of 4096 bytes; malformed JSON ignored; host-only commands checked on the server.
Memory exhaustion	Rooms and audio deleted 60 s after becoming empty; one audio file per room, replaced on re-upload.
Privacy	No accounts, cookies or personal data; audio is not written to disk.
Copyright	The host supplies their own file for a private listening room; the server does not index, publish or retain tracks.

13. Deployment Plan

- Deploy the repository as one web service (Render, Railway or Fly.io) with start command 'npm start'; the platform provides the PORT environment variable and HTTPS.
- HTTPS is required in practice: mobile browsers restrict some audio features on insecure origins, and WSS is needed when the page is served over HTTPS.
- No environment variables other than PORT are needed. No database service is provisioned.
- Vercel is not suitable for the server because its serverless functions do not keep WebSocket connections or in-memory state; this is why the whole project runs as one long-lived Node process.
- Free tiers may sleep after inactivity (first request can take 30 to 60 seconds) and restarts erase in-memory rooms; this is documented as a known limitation.

14. Testing and Evaluation Plan

14.1 Functional test cases

ID	Scenario	Expected result	Result
TC01	Create a room	201 with 6-char code and host token	
TC02	Join with a valid code	Connects; listener count increases	
TC03	Join with an invalid code	Error message; WebSocket closes with 4004	

ID	Scenario	Expected result	Result
TC04	Upload a non-audio file	415 and error shown	
TC05	Upload a file larger than 25 MB	Rejected (413) and message shown	
TC06	Upload with wrong host token	403	
TC07	Play with all clients ready	All devices start together after about 1 s	
TC08	A new listener is still loading	Play stays disabled until it is ready	
TC09	Pause, resume, seek	All devices follow and stay aligned	
TC10	Join while the song is playing	Late joiner starts at the correct position	
TC11	Turn Wi-Fi off and on	Client reconnects, re-syncs and resumes	
TC12	Move delay slider with Bluetooth	Audible alignment improves	
TC13	Host reloads the page	Host rights restored from stored token	
TC14	Everyone leaves	Room and audio removed after 60 s	
TC15	Create 11 rooms in an hour from one IP	11th request returns 429	
TC16	Join on iPhone Safari	Audio unlocked by Join button and plays	
TC17	Send a WebSocket message above 4096 bytes	Connection closed by server	

14.2 Synchronization accuracy experiment

Method. Place two phones side by side and have both join one room. Use a short audio file made of sharp clicks at one-second intervals. Record both phones with a third device (a laptop microphone and Audacity work well). In the waveform, measure the time difference between the two copies of each click. Repeat ten times, including trials on Wi-Fi and on mobile data, and with a restart in between to include fresh clock estimates. Report the mean, maximum and standard deviation, and note the reported sync bound shown in the UI.

Trial	Network	UI sync bound (ms)	Measured offset between phones (ms)
1	Wi-Fi		
2	Wi-Fi		
3	Wi-Fi		
4	Wi-Fi		
5	Wi-Fi		
6	4G		
7	4G		
8	4G		
9	4G		
10	4G		
Summary		mean:	mean / max / std dev:

These tables are intentionally blank; they will be filled with measured data and included in the final report.

14.3 Other checks

- Cross-browser check: Chrome on Android, Safari on iPhone, desktop Chrome and Firefox.
- Soak test: play a 10-minute track and log how often drift correction triggers.
- Load check: several simultaneous rooms with multiple listeners each; note memory use.

- Failure injection: kill the server mid-song and confirm the client reports the problem and recovers.

15. Risks, Limitations and Mitigations

Risk or limitation	Impact	Mitigation
Bluetooth latency not detectable	Audible offset on wireless headphones	Manual delay slider; document recommended calibration.
iOS Safari audio restrictions	No sound until user gesture	Join the party button creates and resumes the AudioContext.
Network asymmetry	Clock estimate error up to RTT/2	Lowest-RTT sample; periodic re-sync; show bound in UI.
Large file on slow network	Long wait before everyone is ready	25 MB cap; per-listener ready count; Play gated on readiness.
Free hosting sleep or restart	Rooms lost, slow first load	Document limitation; optional Redis in future work.
Memory held per room	Server memory grows with large uploads	Size limit; delete 60 s after empty; one file per room.
Different audio decoders per device	Rare decode failures for unusual codecs	Recommend mp3, m4a or wav; show error message on failure.

16. Work Plan

16.1 Initial build (one intensive session)

Phase	Work	Output
1	Project setup, Express server, room create/join API, WebSocket rooms	Rooms working with two clients
2	Clock sync handshake and sync indicator	Offset and bound displayed
3	Audio upload, download, decode, scheduled playback, host controls	Two devices play in sync
4	Drift correction, reconnect, late join, delay slider	Robust playback
5	Responsive polish, error states, deployment, README	Public URL

16.2 Evaluation and report phase

Week	Activity
1	Run functional test cases TC01 to TC17 on real devices; fix defects.
1	Run the accuracy experiment (Section 14.2); collect data.
2	Write final report with results, screenshots, architecture and algorithm diagrams.
2	Record a demo video; prepare slides and live demo script.
2	Final deployment check and submission.

17. Expected Deliverables and Learning Outcomes

Deliverables

- Complete source code in one repository with a clear README and setup instructions.
- Deployed application on a public HTTPS URL.
- This proposal and a final project report containing measured results.
- Architecture and algorithm diagrams, test report and screenshots.

- Demonstration video and a live demo using two or more phones.

Learning outcomes

- Practical understanding of clock synchronization and its error bounds.
- Experience designing a REST plus WebSocket protocol and shared state on the server.
- Working knowledge of the Web Audio API and browser autoplay policies.
- Handling of failure cases: reconnection, late joiners, partial readiness.
- Security basics for public web services: tokens, validation, rate limiting, resource limits.
- Deployment of a real-time Node.js application and honest documentation of its limits.

18. Future Enhancements

- Redis for room state so rooms survive restarts and the app can run on several instances.
- Shared queue or playlist with host-approved suggestions from listeners.
- Host handover if the host disconnects.
- Automatic latency calibration using a microphone-based click test on each device.
- Chunked or streamed audio delivery to reduce time-to-ready for large files.
- Optional accounts and saved rooms, backed by a database such as PostgreSQL.
- Progressive web app packaging and a QR code for joining.

19. References

- D. Mills et al., *Network Time Protocol Version 4: Protocol and Algorithms Specification*, RFC 5905, IETF, 2010.
- I. Fette and A. Melnikov, *The WebSocket Protocol*, RFC 6455, IETF, 2011.
- F. Cristian, "Probabilistic clock synchronization," *Distributed Computing*, vol. 3, no. 3, pp. 146-158, 1989.
- W3C, *Web Audio API* specification (AudioContext, AudioBufferSourceNode).
- Express.js documentation (routing, static files, express.raw body parser).
- ws: a Node.js WebSocket library, project documentation.
- MDN Web Docs: Web Audio API, WebSocket API, Autoplay guide for media and Web Audio APIs.

Appendix A. Running the Project Locally

```
cd silent-disco
npm install
npm start            # http://localhost:3000
npm run dev          # restarts on file changes (Node 18+)
```

To test on phones, connect them to the same Wi-Fi as the computer and open http://COMPUTER-IP:3000. Note that some browser audio features need HTTPS, so final testing should use the deployed URL.

Appendix B. Demo Script (about 3 minutes)

- Open the site on a laptop and click Start a party; show the room code and link.
- Open the link on two phones and tap Join the party; show the listener and ready counts.
- Upload a song as host; show everyone turning ready.

- Press Play and let the teacher hear the phones side by side; show the sync bound in the UI.
- Seek and pause; join with a third phone mid-song to demonstrate late join.
- Move the delay slider while explaining Bluetooth latency; show the measured accuracy table.